

Rapport Deep Learning - Projet jeu de GO

Tomil Shi, Emile Descroix, Alexis Michel

Mars 2026

Sommaire

1	Introduction	3
1.1	Contexte du projet	3
1.2	Contraintes du projet	3
2	Description des données	3
2.1	Format des entrées	3
2.2	Format des sorties	3
2.3	Chargement des données	4
3	Architecture du réseau	4
3.1	Bloc d'entrée	4
3.2	Tronc résiduel : 7 blocs	4
3.3	Bloc Squeeze-and-Excitation (SE)	5
3.4	Policy Head	5
3.5	Value Head	5
4	Stratégie d'entraînement	5
4.1	Data augmentation	5
4.2	Fonction de perte	6
4.3	Optimiseur et planning du learning rate	7
4.4	Protocole d'entraînement	7
5	Résultats	7
5.1	Performances finales	8
5.2	Performances en validation	8
5.3	Interprétation	8

1 Introduction

1.1 Contexte du projet

Le jeu de Go est l'un des jeux de stratégie les plus complexes qui existent. Sur un plateau de 19×19 intersections, deux joueurs placent alternativement des pierres noires et blanches en cherchant à contrôler le plus grand territoire possible. L'espace de recherche est colossal : environ $2,1 \times 10^{170}$ positions légales, soit bien davantage que le nombre d'atomes dans l'univers observable.

Ce projet s'inscrit dans la continuité des travaux pionniers d'AlphaGo (DeepMind, 2016), qui a démontré la capacité des réseaux de neurones convolutifs à apprendre une politique de jeu compétitive. L'objectif ici est de concevoir et d'entraîner un réseau de neurones capable de prédire simultanément :

- **Policy** : la probabilité de jouer en chacune des 361 intersections du plateau;
- **Value** : la probabilité de victoire du joueur blanc étant donné l'état actuel de la partie.

1.2 Contraintes du projet

Le projet impose une contrainte stricte sur la taille du modèle : le nombre total de paramètres entraînaables doit rester **strictement inférieur à 100 000**. Cette contrainte oblige à concevoir une architecture efficace, capable de tirer le maximum de chaque paramètre disponible.

Les données d'entraînement proviennent de 1 000 000 parties auto-jouées par le programme KataGo, un des moteurs de Go les plus forts au monde. Chaque position est encodée sous la forme d'un tenseur de dimensions $19 \times 19 \times 31$, soit 31 plans binaires décrivant l'état du plateau.

2 Description des données

2.1 Format des entrées

Chaque position de jeu est représentée par un tenseur de forme $(19, 19, 31)$. Les 31 plans d'entrée encodent les informations stratégiques et tactiques nécessaires au modèle:

Plan(s)	Description
Plan 0	Couleur du joueur actif (1 = Noir, 0 = Blanc)
Plans 1–2	État actuel du plateau (pierres noires / pierres blanches)
Plans 3–6	Deux états précédents du plateau (2×2 couleurs)
Plans 7–30	Ladders et autres caractéristiques tactiques calculées

Table 1: Répartition des 31 plans d'entrée du tenseur.

2.2 Format des sorties

Le réseau doit prédire deux objectifs distincts :

- **Policy** : un vecteur de taille 361 (19×19), encodé en one-hot où la valeur 1.0 est placée sur l'intersection jouée. Cette cible est entraînée avec une fonction de perte categorical crossentropy.
- **Value** : un scalaire binaire, proche de 1.0 si les Blancs gagnent et de 0.0 si les Noirs gagnent. Cette cible est entraînée avec une binary crossentropy.

2.3 Chargement des données

Le chargement est assuré par le module golois, une bibliothèque C compilée qui expose deux fonctions clés:

- `getBatch` : permet de charger un batch de N positions à partir d'un index précis (offset) dans le fichier `games.data`.
- `getValidation` : charge un jeu de données de validation fixe, permettant d'évaluer les performances du modèle de manière reproductible entre différents entraînements.

3 Architecture du réseau

Le réseau suit la structure classique des réseaux de type AlphaZero : un tronc commun suivi de deux têtes spécialisées. La contrainte de 100 000 paramètres impose d'utiliser des convolutions séparables en profondeur (`SeparableConv2D`) plutôt que des `Conv2D` classiques, ce qui réduit le coût par couche d'un facteur ≈ 10 .

3.1 Bloc d'entrée

La première couche est une `Conv2D(64, 3×3)` standard. Ce choix est délibéré : les convolutions séparables ne sont efficaces que lorsque les canaux d'entrée véhiculent des informations indépendantes. Or, les 31 plans d'entrée sont fortement corrélés (état actuel, historique, ladders), et une convolution standard mélange mieux ces informations initiales.

Cette couche est suivie d'une `BatchNormalization` et d'une activation `ReLU`, conformément à la pratique standard pour les réseaux profonds.

3.2 Tronc résiduel : 7 blocs

Le tronc est composé de 7 blocs résiduels identiques. Chaque bloc implémente le schéma suivant :

Étape	Opération	Paramètres approx.
1	<code>SeparableConv2D(64, 3×3) + BN + ReLU</code>	$\approx 4\,600$
2	<code>SeparableConv2D(64, 3×3) + BN</code>	$\approx 4\,600$
3	SE block (ratio = 8)	$\approx 1\,096$
4	Add (connexion résiduelle) + <code>ReLU</code>	0

Table 2: Structure d'un bloc résiduel séparable avec SE block.

Connexions résiduelles. Inspirées de ResNet, elles permettent au gradient de remonter jusqu'aux premières couches sans vanish, rendant possible l'entraînement de réseaux plus profonds.

SeparableConv2D. Une convolution séparable décompose l'opération en deux étapes : une convolution depthwise (filtre spatial indépendant par canal) suivie d'une convolution pointwise (1×1 , mélange des canaux). Le rapport du nombre de paramètres par rapport à une `Conv2D` classique est :

$$\frac{1}{N} + \frac{1}{k^2} \approx \frac{1}{9} \quad \text{pour } k = 3, N = 64$$

soit environ 9 fois moins de paramètres pour des filtres 3×3 .

3.3 Bloc Squeeze-and-Excitation (SE)

Le SE block recalibre dynamiquement l'importance de chaque canal de feature maps en fonction du contenu global de la position. Il opère en trois étapes :

1. **Squeeze** : un `GlobalAveragePooling2D` condense chaque carte de features (19×19) en un scalaire, produisant un vecteur $\mathbf{s} \in \mathbb{R}^{64}$;
2. **Excitation** : deux couches denses ($64 \rightarrow 8 \rightarrow 64$) apprennent un vecteur de pondération $\mathbf{e} \in [0, 1]^{64}$ via une activation sigmoïde ;
3. **Recalibrage** : les feature maps sont multipliées canal par canal par $\mathbf{e}$.

Formellement, le recalibrage s'écrit :

$$\tilde{\mathbf{x}}_c = e_c \cdot \mathbf{x}_c, \quad c = 1, \dots, C$$

où $\mathbf{x}_c$ est la c -ième carte de features et e_c son poids appris. Le coût paramétrique par SE block est :

$$\left(C \times \frac{C}{r} + \frac{C}{r}\right) + \left(\frac{C}{r} \times C + C\right) = (64 \times 8 + 8) + (8 \times 64 + 64) = 520 + 576 = 1096$$

soit un surcoût total de $7 \times 1096 = 7672$ paramètres pour les 7 blocs.

3.4 Policy Head

La policy head produit une distribution de probabilité sur les 361 coups possibles :

$$\mathbf{p} = \text{softmax}(f_\theta(\mathbf{x})) \in \Delta^{360}$$

Son architecture utilise deux convolutions successives avant le softmax :

$$\text{Conv2D}(4, 1 \times 1) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv2D}(1, 1 \times 1) \rightarrow \text{Flatten} \rightarrow \text{Softmax}$$

Le premier `Conv2D(4)` permet au réseau d'apprendre des combinaisons non-linéaires entre les 64 canaux du tronc avant de produire la heatmap finale à 1 canal.

3.5 Value Head

La value head prédit $v \in [0, 1]$, la probabilité de victoire du joueur blanc :

$$\text{Conv2D}(32, 1 \times 1) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{GAP} \rightarrow \text{Dense}(64) \rightarrow \text{Dense}(1, \sigma)$$

Le `GlobalAveragePooling2D` (GAP) est préféré au `Flatten` : il remplace les $32 \times 19 \times 19 = 11\,552$ valeurs par 32 moyennes, économisant plusieurs milliers de paramètres dans la couche dense suivante.

4 Stratégie d'entraînement

4.1 Data augmentation

Le Go possède une propriété fondamentale : le jeu est invariant par rotation à 90° et par symétrie horizontale. Un plateau retourné ou pivoté représente exactement la même situation stratégique. Ces transformations forment le groupe diédral D_4 , qui définit 8 transformations équivalentes :

Composant	Paramètres approx.	% du total
Bloc d'entrée Conv2D(64, 3×3)	18 112	≈ 18%
7 blocs résiduels séparables (dont SE)	76 664	≈ 77%
policy head	276	≈ 0.2%
value head	4 353	≈ 4%
Total	99 405	100%

Table 3: Répartition du budget paramètres entre les composants du réseau.

$$\mathcal{T} = \left\{ r^k \circ s^f \mid k \in \{0, 1, 2, 3\}, f \in \{0, 1\} \right\}$$

où r désigne une rotation de 90° et s une symétrie horizontale. À chaque epoch, une transformation $t \sim \mathcal{U}(\mathcal{T})$ est tirée aléatoirement et appliquée simultanément au plateau d'entrée et à la politique :

```

k      = randint(0, 4)                # rotation : 0°, 90°, 180° ou 270°
flip   = randint(0, 2)                # symétrie horizontale
x_aug  = rot90(x, k, axes=(1,2))      # rotation du plateau
p_aug  = rot90(p.reshape(-1,19,19), k)
        .reshape(-1, 361)            # rotation de la policy
if flip:
    x_aug = flip(x_aug, axis=2)
    p_aug = flip(p_aug.reshape(-1,19,19), axis=2)
        .reshape(-1, 361)

```

La valeur (value) est un scalaire global indépendant de l'orientation du plateau : elle est conservée sans modification. Cette augmentation multiplie effectivement par 8 la diversité des positions vues, sans aucun coût en paramètres ni en données supplémentaires.

4.2 Fonction de perte

Le modèle est entraîné avec une perte composite sur les deux head :

$$\mathcal{L} = w_p \cdot \mathcal{L}_{\text{policy}} + w_v \cdot \mathcal{L}_{\text{value}}$$

Policy loss : La politique cible est encodée en one-hot, ce qui conduit naturellement à une entropie croisée catégorielle :

$$\mathcal{L}_{\text{policy}} = - \sum_{i=1}^{361} y_i \log \hat{p}_i$$

Value loss : La valeur cible est binaire ($y_v \in \{0, 1\}$), ce qui justifie l'utilisation d'une entropie croisée binaire plutôt qu'un MSE, car elle pénalise davantage les prédictions très confiantes et erronées :

$$\mathcal{L}_{\text{value}} = - [y_v \log \hat{v} + (1 - y_v) \log(1 - \hat{v})]$$

Pondération : Les poids $w_p = 2,0$ et $w_v = 1,0$ reflètent la priorité accordée à la précision du coup joué, qui est la métrique principale d'évaluation du projet.

4.3 Optimiseur et planning du learning rate

L’optimiseur Adam est utilisé avec un learning rate initial $\eta_0 = 10^{-3}$. Pour un entraînement long sur 3 000 epochs, un Cosine Decay est appliqué. En effet, ce dernier surveille une métrique qui fluctue artificiellement à chaque epoch (la distribution des données change à chaque appel de `getBatch`), ce qui peut provoquer des réductions prématurées et injustifiées du learning rate. Le Cosine Decay suit :

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T}\right) \right)$$

où T est le nombre total de steps d’entraînement et $\eta_{\min} = 10^{-6}$ est le learning rate minimal atteint en fin d’entraînement. Cette stratégie assure une descente continue et régulière, favorisant une convergence progressive sans discontinuités.

4.4 Protocole d’entraînement

L’entraînement est organisé en epochs de $N = 10\,000$ positions. À chaque epoch $i \in \{1, \dots, 3000\}$:

1. Un nouveau batch est chargé depuis `games.data` via `golois.getBatch(..., i × N)` ;
2. La data augmentation est appliquée (transformation aléatoire $t \sim \mathcal{U}(\mathcal{T})$) ;
3. Le modèle est entraîné pendant 1 époque sur ce batch (`batch_size = 256`) ;
4. Tous les 10 epochs, `gc.collect()` libère la mémoire Python ;
5. Tous les 50 epochs, une validation complète est effectuée via `golois.getValidation` et le modèle est sauvegardé au format `.h5`.

Au total, $3\,000 \times 10\,000 = 30$ millions de positions sont vues durant l’entraînement, soit 30 fois la taille du jeu de données disponible. Le tableau 4 récapitule les hyperparamètres utilisés.

Hyperparamètre	Valeur
Taille du epoch (N)	10 000
Taille du batch	256
Nombre de epochs	3 000
Learning rate initial	10^{-3}
Learning rate minimal	10^{-6}
Schedule	Cosine Decay
Optimiseur	Adam ($\beta_1 = 0,9$, $\beta_2 = 0,999$)
Poids policy / value	2,0 / 1,0
Régularisation L2	10^{-4} (toutes les couches)
Data augmentation	D_4 (8 transformations)

Table 4: Récapitulatif des hyperparamètres d’entraînement.

5 Résultats

L’entraînement du modèle s’est étendu sur 3 000 epochs de 10 000 positions chacun, soit un total de 30 millions de positions traitées avec augmentation de données. Les performances finales démontrent l’efficacité de l’architecture résiduelle sous contrainte de paramètres.

5.1 Performances finales

Au cours du dernier epoch (3000), le modèle a traité les 10 000 dernières positions avec les scores suivants en fin d'époque :

- **Loss totale** : 4,7947
- **Policy Accuracy (Train)** : 44,90 %
- **Value Loss** : 0,5774
- **Value MAE (Train)** : 0,1985

5.2 Performances en validation

Au terme du dernier epoch d'entraînement, le modèle atteint les métriques de validation suivantes:

- **Loss totale** : 4,8196
- **Policy Accuracy (Validation)** : 45,40 %
- **Value MAE (Validation)** : 0,2268

Métrique	Baseline	Modèle Final
Nombre de paramètres	65 479	99 405
Policy Accuracy (Val)	17,8 %	45,40 %
Value MAE (Val)	0,292	0,2268

Table 5: Comparaison des performances finales contre le modèle de référence.

5.3 Interprétation

L'augmentation massive de la policy accuracy (passant de 17,8 % à 45,40 %) valide les choix architecturaux, notamment l'utilisation des convolutions séparables qui ont permis d'augmenter la profondeur du réseau (7 blocs résiduels) tout en respectant la limite des 100 000 paramètres.

On observe également que la précision en validation (45,40 %) est légèrement supérieure à celle de l'entraînement (44,90 %), ce qui suggère une excellente capacité de généralisation du modèle, sans signe de surapprentissage (overfitting), grâce à la stratégie d'augmentation de données par symétrie D_4 .